

NQueen

Sunday, 10 August 2025 8:08 AM

Java

```
package GreatLearning;
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
public class NQueens {
public static List< List< String >> solveNQueens(int n) {
    // Create an n x n character array to represent the chessboard.
    char[][] board = new char[n][n];
    // Initialize the board with '.' to represent empty squares.
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            board[i][j] = '.';
    // This list will store all the valid solutions. Each solution is a list of strings.
    List< List< String >> res = new ArrayList< List< String >> ();
    // This array tracks if a row is under attack.
    int leftRow[] = new int[n];
    // This array tracks if an upper diagonal is under attack.
    // The size is 2*n-1 because there are that many unique upper diagonals.
    int upperDiagonal[] = new int[2 * n - 1];
    // This array tracks if a lower diagonal is under attack.
    // The size is 2*n-1 because there are that many unique lower diagonals.
    int lowerDiagonal[] = new int[2 * n - 1];
    // Start the recursive backtracking process from the first column (column 0).
    solve(0, board, res, leftRow, lowerDiagonal, upperDiagonal);
    // Return the list of all found solutions.
    return res;
}

static void solve(int col, char[][] board, List< List< String >> res, int leftRow[], int lowerDiagonal[], int upperDiagonal[]) {
    // Base case: if all columns have been processed (a Queen has been placed in each),
    // a valid solution has been found.
    if (col == board.length) {
        // Add the current board configuration to the results list.
        res.add(construct(board));
        return;
    }
    // Iterate through each row of the current column to try placing a Queen.
    for (int row = 0; row < board.length; row++) {
        // Check if the current position (row, col) is safe to place a Queen.
        // It's safe if the row, lower diagonal, and upper diagonal are not under attack.
        if (leftRow[row] == 0 && lowerDiagonal[row + col] == 0 && upperDiagonal[board.length - 1 + col - row] == 0) {
            // Place a Queen at the current position.
            board[row][col] = 'Q';
            // Mark the row and both diagonals as under attack.
            leftRow[row] = 1;
            lowerDiagonal[row + col] = 1;
            upperDiagonal[board.length - 1 + col - row] = 1;
            // Recursively call solve for the next column.
            solve(col + 1, board, res, leftRow, lowerDiagonal, upperDiagonal);
            // Backtrack: remove the Queen to explore other possibilities.
            board[row][col] = '.';
            // Unmark the row and diagonals.
            leftRow[row] = 0;
            lowerDiagonal[row + col] = 0;
            upperDiagonal[board.length - 1 + col - row] = 0;
        }
    }
}

static List< String > construct(char[][] board) {
    // Create a list of strings to represent the board for a single solution.
    List< String > res = new LinkedList< String > ();
    // Convert each row of the character array into a string and add it to the list.
    for (int i = 0; i < board.length; i++) {
        String s = new String(board[i]);
        res.add(s);
    }
    // Return the list of strings representing the solution.
    return res;
}
}
```